

AI TRAINING – WEEK 2

DAY 4 ASSIGNMENT

OOP Basics: Classes, Objects, Constructors, Methods & Inheritance

Plan → Create Classes → Use Objects → Reuse Code

1. What You Will Learn

You will learn the basics of Object-Oriented Programming (OOP) in Python. You will create classes and objects, initialize object data using constructors, define methods, and understand how inheritance helps reuse code.

- Classes – define the structure and behavior of objects
- Objects – create individual instances of a class
- Constructors – initialize object attributes using `__init__()`
- Attributes – store information inside an object
- Methods – define actions that objects can perform
- Inheritance – create a child class from an existing parent class
- Use `self` to access the current object's attributes and methods
- Build a small class-based application

2. Student Deliverable – Keep It Simple

For this assignment, your main submission is ONE Python program that demonstrates the OOP concepts.

Main file: `oop_practice_YourName.py`

Keep the program and any supporting Python files in the same project folder.

- Create at least two classes
- Create and use objects
- Use constructors to initialize attributes
- Write methods that perform useful actions
- Demonstrate simple inheritance
- Test the program with different object values
- Use clear names and readable code

No screenshots or separate Word/PDF submission is required unless your trainer asks for them.

3. Part A – Class and Object Basics

Create a class called `Student`. Give it a name and age. Create two `Student` objects with different values.

- Print each student's name and age.
- Change one object's name and print it again.
- Observe that changing one object does not automatically change the other object.

Example

```
class Student:
    pass
student1 = Student()
student2 = Student()
student1.name = "Rahul"
student1.age = 21
student2.name = "Priya"
```

```

student2.age = 22
print(student1.name, student1.age)
print(student2.name, student2.age)

```

Task: Rewrite the example so name and age are initialized through a constructor instead of being assigned after object creation.

```

class Student:
    def __init__(self, name, age):
        self.name = name
        self.age = age
student1 = Student("Rahul", 21)
student2 = Student("priya", 22)
print(student1.name, student1.age)
print(student2.name, student2.age)
student1.name = "lasya"
print("After changing name:")
print(student1.name, student1.age)
print(student2.name, student2.age)

```

OUTPUT:

```

Rahul 21
priya 22
After changing name:
lasya 21
priya 22

```

4. Part B – Constructors and Attributes

A constructor is commonly written using `__init__()`. It runs when an object is created and can assign initial values to that object.

- Create a class called Mobile with brand, model, and price attributes.
- Use `__init__()` to initialize all three attributes.
- Create at least three Mobile objects with different values.
- Print the details of every object.

Example

```

class Mobile:
    def __init__(self, brand, model, price):
        self.brand = brand
        self.model = model
        self.price = price
phone1 = Mobile("Samsung", "A55", 35000)
phone2 = Mobile("Apple", "iPhone 15", 65000)
print(phone1.brand, phone1.model, phone1.price)
print(phone2.brand, phone2.model, phone2.price)

```

- Add a new attribute called storage and display it.
- Try creating an object without passing a required value. Observe the error and explain why it happens.

CODE:

```

class Mobile:
    def __init__(self, brand, model, price):
        self.brand = brand

```

```

        self.model = model
        self.price = price
# Create three Mobile objects
phone1 = Mobile("Samsung", "A55", 35000)
phone2 = Mobile("Apple", "iPhone 15", 65000)
phone3 = Mobile("Vivo", "V27", 30000)
# Print details of all objects
print("Mobile 1:", phone1.brand, phone1.model, phone1.price)
print("Mobile 2:", phone2.brand, phone2.model, phone2.price)
print("Mobile 3:", phone3.brand, phone3.model, phone3.price)
# Add a new attribute called storage
phone1.storage = "128GB"
print("Phone 1 Storage:", phone1.storage)
# Change the price of one object
phone1.price = 36000
print("Updated Samsung Price:", phone1.price)
# Try creating an object without a required value
try:
    phone4 = Mobile("Samsung", "S24")
except TypeError as e:
    print("Error:", e)

```

OUTPUT:

Mobile 1: Samsung A55 35000

Mobile 2: Apple iPhone 15 65000

Mobile 3: Vivo V27 30000

Phone 1 Storage: 128GB

Updated Samsung Price: 36000

Error: Mobile.__init__() missing 1 required positional argument: 'price'

5. Part C – Methods

Methods are functions defined inside a class. They commonly use self to access the current object's attributes.

- Create a class called BankAccount with account_holder and balance.
- Add a display_balance() method.
- Add a deposit(amount) method that increases the balance.
- Add a withdraw(amount) method that decreases the balance only when enough money is available.
- Create an object and test deposit and withdrawal operations.

Example

```

class BankAccount:
    def __init__(self, account_holder, balance):
        self.account_holder = account_holder
        self.balance = balance
    def display_balance(self):
        print("Account holder:", self.account_holder)
        print("Balance:", self.balance)
    def deposit(self, amount):
        self.balance += amount

```

```

account = BankAccount("Rahul", 1000)
account.display_balance()
account.deposit(500)
account.display_balance()

```

CODE:

```

class BankAccount:
    def __init__(self, account_holder, balance):
        self.account_holder = account_holder
        self.balance = balance
    def display_balance(self):
        print("Account holder:", self.account_holder)
        print("Balance:", self.balance)
    def deposit(self, amount):
        if amount <= 0:
            print("Deposit amount must be positive.")
        else:
            self.balance += amount
            print("Deposit successful:", amount)
    def withdraw(self, amount):
        if amount <= 0:
            print("Withdrawal amount must be positive.")
        elif amount > self.balance:
            print("Insufficient balance.")
        else:
            self.balance -= amount
            print("Withdrawal successful:", amount)

```

```

# Create an object
account = BankAccount("Rahul", 1000)
# Display initial balance
account.display_balance()
# Deposit money
account.deposit(500)
account.display_balance()
# Try a negative deposit
account.deposit(-200)
# Withdraw money
account.withdraw(300)
account.display_balance()
# Try withdrawing more than the balance
account.withdraw(2000)
# Display final balance
account.display_balance()

```

OUTPUT:

```

Account holder: Rahul
Balance: 1000
Deposit successful: 500
Account holder: Rahul
Balance: 1500
Deposit amount must be positive.
Withdrawal successful: 300
Account holder: Rahul
Balance: 1200
Insufficient balance.
Account holder: Rahul
Balance: 1200

```

Challenge: Add validation so a negative deposit is rejected. Also handle a withdrawal request that is greater than the current balance.

CODE:

```

class BankAccount:
    def __init__(self, account_holder, balance):
        self.account_holder = account_holder

```

```

        self.balance = balance
    def display_balance(self):
        print("Account holder:", self.account_holder)
        print("Balance:", self.balance)
    def deposit(self, amount):
        if amount <= 0:
            print("Positive deposit amount required.")
        else:
            self.balance += amount
            print("Deposit successful.")
    def withdraw(self, amount):
        if amount <= 0:
            print("Positive withdrawal amount required.")
        elif amount > self.balance:
            print("Insufficient balance.")
        else:
            self.balance -= amount
            print("Withdrawal successful.")
account = BankAccount("Rahul", 1000)
account.display_balance()
account.deposit(-200)
account.deposit(500)
account.display_balance()
account.withdraw(1000)
account.withdraw(300)
account.display_balance()

```

OUTPUT:

```

Account holder: Rahul
Balance: 1000
Negative deposit is not allowed.
Deposit successful.
Account holder: Rahul
Balance: 1500
Withdrawal successful.
Withdrawal successful.
Account holder: Rahul
Balance: 200

```

6. Part D – Methods That Return Values

A method can print a result or return a value using return. Practice returning values from methods.

- Create a Rectangle class with length and width.
- Add an area() method that returns length multiplied by width.

- Add a `perimeter()` method that returns 2 multiplied by the sum of length and width.
- Create two `Rectangle` objects and print their areas and perimeters.

Example

```
class Rectangle:
    def __init__(self, length, width):
        self.length = length
        self.width = width
    def area(self):
        return self.length * self.width
    def perimeter(self):
        return 2 * (self.length + self.width)
shape = Rectangle(10, 5)
print("Area:", shape.area())
print("Perimeter:", shape.perimeter())
CODE:
class Rectangle:
    def __init__(self, length, width):
        self.length = length
        self.width = width
    def area(self):
        return self.length * self.width
    def perimeter(self):
        return 2 * (self.length + self.width)
# Create two Rectangle objects
r1 = Rectangle(5, 4)
r2 = Rectangle(10, 12)
# Print area and perimeter of the first rectangle
print("Rectangle 1 Area:", r1.area())
print("Rectangle 1 Perimeter:", r1.perimeter())
# Print area and perimeter of the second rectangle
print("Rectangle 2 Area:", r2.area())
print("Rectangle 2 Perimeter:", r2.perimeter())
OUTPUT:
Rectangle 1 Area: 20
Rectangle 1 Perimeter: 18
Rectangle 2 Area: 120
Rectangle 2 Perimeter: 44
```

7. Part E – Inheritance Overview

Inheritance allows a child class to reuse attributes and methods from a parent class. A child class can also define its own methods.

- Create a parent class called `Person` with name and age.
- Add a `introduce()` method that prints the person's details.
- Create a child class called `Employee` that inherits from `Person`.
- Add an `employee_id` attribute to `Employee`.
- Create an `Employee` object and call both the inherited method and a method of its own.

Example

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def introduce(self):
        print("Name:", self.name)
        print("Age:", self.age)

class Employee(Person):
```

```

def __init__(self, name, age, employee_id):
    super().__init__(name, age)
    self.employee_id = employee_id

def show_employee_id(self):
    print("Employee ID:", self.employee_id)

employee = Employee("Ananya", 23, "E101")
employee.introduce()
employee.show_employee_id()

```

Tasks:

- Explain what Person and Employee represent in this example.
- Explain why super().__init__(name, age) is used.
- Create another child class called Trainer with a subject attribute.
- Try calling a method that exists only in the child class from a parent-class object. Record what happens.

Code:

```

class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age
    def introduce(self):
        print("Name:", self.name)
        print("Age:", self.age)

class Employee(Person):
    def __init__(self, name, age, employee_id):
        super().__init__(name, age)
        self.employee_id = employee_id
    def show_employee_id(self):
        print("Employee ID:", self.employee_id)

class Trainer(Person):
    def __init__(self, name, age, subject):
        super().__init__(name, age)
        self.subject = subject
    def show_subject(self):
        print("Subject:", self.subject)

# Create an Employee object
employee = Employee("Ananya", 23, "E101")
employee.introduce()
employee.show_employee_id()

# Create a Trainer object
trainer = Trainer("Rahul", 30, "Python")
trainer.introduce()
trainer.show_subject()

# Try calling a child-class method from a parent object
person = Person("Priya", 25)
try:

```

```
    person.show_employee_id()
except AttributeError as e:
    print("Error:", e)
```

OUTPUT:

Name: Ananya

Age: 23

Employee ID: E101

Name: Rahul

Age: 30

Subject: Python

Error: 'Person' object has no attribute 'show_employee_id'

1. What do Person and Employee represent?

- Person is the parent class containing common details like name and age.
- Employee is the child class that inherits from Person and adds an employee ID.

2. Why do we use super().__init__(name, age)?

It calls the parent class constructor to initialize the name and age.

3. What is the Trainer class?

Trainer is another child class of Person. It inherits name and age and adds a subject attribute.

4. What happens when we call a child-class method from a parent object?

Python raises an AttributeError because the parent object does not have access to methods defined only in the child class.

8. Part F – Organizing OOP Code into Modules

A module is a Python file that can contain classes and functions. Practice placing a class in one file and importing it into another.

Suggested structure:

```
oop_project/
  main.py
  student.py
```

- In student.py, create a Student class with a constructor and a display_details() method.
- In main.py, import Student from student.py.
- Create at least two Student objects and call display_details() for each.

```
# student.py
class Student:
    def __init__(self, name, course):
        self.name = name
        self.course = course
    def display_details(self):
        print("Name:", self.name)
        print("Course:", self.course)

# main.py
from student import Student
student1 = Student("Rahul", "Python")
student2 = Student("Priya", "Cyber Security")
student1.display_details()
student2.display_details()
```

Keep both files in the same folder for this basic example.

Output:

Name: Rahul

Course: Python
Name: Priya
Course: Cyber Security

9. Part G – Quick Debugging

Find the problem, fix it, run the code, and explain the mistake.

Debugging Problem 1

```
class Student:
    def __init__(self, name):
        name = name
student = Student("Rahul")
print(student.name)
```

Task: Why does this code fail when printing student.name? Fix the constructor.

CODE:

```
class Student:
    def __init__(self, name):
        self.name = name
student = Student("Rahul")
print(student.name)
```

OUTPUT:

Rahul

Explanation: name = name only assigns a local variable to itself. Using self.name = name stores the name in the object.

Debugging Problem 2

```
class Car:
    def __init__(self, brand):
        self.brand = brand

car = Car()
print(car.brand)
```

Task: Identify the problem and fix the object creation or constructor.

Code:

```
class Car:
    def __init__(self, brand):
        self.brand = brand

car = Car("Toyota")
print(car.brand)
```

OUTPUT:

Toyota

Explanation: The constructor requires a brand argument, but Car() was called without one. Passing "Toyota" fixes the error.

Debugging Problem 3

```
class Calculator:
    def add(a, b):
        return a + b
calc = Calculator()
print(calc.add(5, 3))
```

Task: Explain the role of self in an instance method and correct the code.

Code:

```
class Calculator:
    def add(self, a, b):
        return a + b
calc = Calculator()
print(calc.add(5, 3))
```

OUTPUT:

8

Explanation: self refers to the current object. Python passes the object automatically when an instance method is called.

Debugging Problem 4

```
class Animal:
    def speak(self):
        print("Animal sound")
class Dog:
    def bark(self):
        print("Woof")
dog = Dog()
dog.speak()
```

Task: The code attempts to call a method that Dog does not have. Use inheritance or change the method call to make it work.

Code:

```
class Animal:
    def speak(self):
        print("Animal sound")
class Dog(Animal):
    def bark(self):
        print("Woof")
dog = Dog()
dog.speak()
dog.bark()
```

OUTPUT:

Animal sound

Woof

Explanation: The original Dog class did not inherit from Animal. Adding (Animal) allows Dog to reuse the parent's speak() method.

10. Part H – Main Hands-on Activity: Student Record System

Build a simple class-based Student Record System. The program should create student objects, store their details, and display useful information.

Each student should have:

- name
- roll_number
- marks

Your program should:

- Create a Student class with a constructor
- Add a display_details() method

- Add a result() method that returns Pass if marks are 40 or above and Fail otherwise
- Create at least four student objects with different marks
- Display each student's details and result
- Find and display the student with the highest marks
- Handle or prevent invalid marks outside the range 0–100
- Use a list to store the student objects

Starter example

```
class Student:
    def __init__(self, name, roll_number, marks):
        self.name = name
        self.roll_number = roll_number
        self.marks = marks

    def display_details(self):
        print(self.name, self.roll_number, self.marks)

    def result(self):
        if self.marks >= 40:
            return "Pass"
        return "Fail"

students = [
    Student("Rahul", 1, 85),
    Student("Priya", 2, 92),
    Student("Arjun", 3, 38),
    Student("Ananya", 4, 76)
]

for student in students:
    student.display_details()
    print("Result:", student.result())
```

Extend the starter example to validate marks and find the highest scorer. You may create helper methods if they make the program clearer.

Bonus: Create a GraduateStudent class that inherits from Student and adds a specialization attribute.

Code:

```
class Student:
    def __init__(self, name, roll_number, marks):
        if not 0 <= marks <= 100:
            raise ValueError("Marks must be between 0 and 100.")
        self.name = name
        self.roll_number = roll_number
        self.marks = marks

    def display_details(self):
        print("Name:", self.name)
        print("Roll Number:", self.roll_number)
        print("Marks:", self.marks)

    def result(self):
        if self.marks >= 40:
            return "Pass"
        return "Fail"
```

Bonus: Child class

```

class GraduateStudent(Student):
    def __init__(self, name, roll_number, marks, specialization):
        super().__init__(name, roll_number, marks)
        self.specialization = specialization
    def display_details(self):
        super().display_details()
        print("Specialization:", self.specialization)
# Create student objects and store them in a list
students = [
    Student("Rahul", 1, 85),
    Student("Priya", 2, 92),
    Student("Arjun", 3, 38),
    Student("Ananya", 4, 76)
]
# Display student details and results
for student in students:
    student.display_details()
    print("Result:", student.result())
    print()
# Find the highest scorer
topper = max(students, key=lambda student: student.marks)
print("Highest Scorer:", topper.name)
print("Highest Marks:", topper.marks)
# Test invalid marks
try:
    invalid_student = Student("Ravi", 5, 120)
except ValueError as e:
    print("Error:", e)
# Test the bonus class
graduate = GraduateStudent("Kiran", 6, 88, "Artificial Intelligence")
print()
graduate.display_details()

```

OUTPUT:

```

Name: Rahul
Roll Number: 1
Marks: 85
Result: Pass
Name: Priya
Roll Number: 2
Marks: 92
Result: Pass
Name: Arjun
Roll Number: 3

```

Marks: 38

Result: Fail

Name: Ananya

Roll Number: 4

Marks: 76

Result: Pass

Highest Scorer: Priya

Highest Marks: 92

Error: Marks must be between 0 and 100.

Name: Kiran

Roll Number: 6

Marks: 88

Specialization: Artificial Intelligence

11. Suggested Program Structure

- Student – stores student details and provides student-related methods
- display_details() – displays one student's information
- result() – returns Pass or Fail
- find_highest_scorer(students) – identifies the student with the highest marks
- main() – creates objects and controls the program

Bonus: Put the Student class in student.py and import it into main.py.

student.py

class Student:

```
def __init__(self, name, roll_number, marks):
    if not 0 <= marks <= 100:
        raise ValueError("Marks must be between 0 and 100.")
    self.name = name
    self.roll_number = roll_number
    self.marks = marks
def display_details(self):
    print("Name:", self.name)
    print("Roll Number:", self.roll_number)
    print("Marks:", self.marks)
def result(self):
    if self.marks >= 40:
        return "Pass"
    return "Fail"
```

main.py

```
from student import Student
def find_highest_scorer(students):
    return max(students, key=lambda student: student.marks)
def main():
```

```

students = [
    Student("Rahul", 1, 85),
    Student("Priya", 2, 92),
    Student("Arjun", 3, 38),
    Student("Ananya", 4, 76)
]
for student in students:
    student.display_details()
    print("Result:", student.result())
    print()
topper = find_highest_scorer(students)
print("Highest Scorer:", topper.name)
print("Highest Marks:", topper.marks)
if __name__ == "__main__":
    main()

```

Output:

```

Name: Rahul
Roll Number: 1
Marks: 85
Result: Pass
Name: Priya
Roll Number: 2
Marks: 92
Result: Pass
Name: Arjun
Roll Number: 3
Marks: 38
Result: Fail
Name: Ananya
Roll Number: 4
Marks: 76
Result: Pass
Highest Scorer: Priya
Highest Marks: 92

```

12. Part I – Explain in Your Own Words

1. What is Object-Oriented Programming (OOP)?
OOP is a programming approach based on classes and objects.
2. What is the difference between a class and an object?
Class: A blueprint used to create objects.
Object: An instance of a class that contains data and uses methods.
3. What is the purpose of `__init__()`?
`__init__()` initializes an object's attributes when it is created.

4. What does self refer to inside an instance method?
self refers to the current object of a class.
5. What is an attribute?
An attribute is a variable that stores data related to an object or class.
6. What is a method?
A method is a function defined inside a class that performs a specific task.
7. What is inheritance?
Inheritance allows a child class to reuse attributes and methods from a parent class.
8. Why do we use super() in a child class?
We use super() to call methods or the constructor of the parent class.
9. What is the difference between print() and return in a method?
print() displays a value on the screen.
return sends a value back to the caller.
10. Why can classes make a program easier to organize?
Classes group related data and methods together, making code easier to reuse and maintain.

13. Daily Deliverable

Submit your working OOP practice program.

File name: oop_practice_YourName.py

Your program should demonstrate:

- Classes and objects
- Constructors and attributes
- Instance methods
- Methods that return values
- Simple inheritance
- Object creation and method calls
- Readable code and basic validation

14. Student Submission Checklist

- ☐ I can explain the difference between a class and an object.✓
- ☐ I created objects from a class.✓
- ☐ I used __init__() to initialize attributes.✓
- ☐ I used self correctly in instance methods.✓
- ☐ I wrote methods that print or return results✓.
- ☐ I understand basic inheritance✓.
- ☐ I used super() in a child class.✓
- ☐ I tested my code with different values✓.
- ☐ I fixed the debugging examples✓.
- ☐ I can explain the main OOP concepts in my own words.✓

15. After This Assignment, You Should Be Able To

- Describe the purpose of OOP in Python.
- Create classes and instantiate objects.

- Initialize object attributes using constructors.
- Write methods that use object data.
- Return calculated values from methods.
- Understand and implement simple inheritance.
- Import a class from another Python module.
- Build a small class-based application.

Key Principle: A class defines a blueprint; objects are individual instances that use its attributes and methods.